DBMS Laboratory Record

KGiSL Institute of Information Management, Coimbatore - 641 035 (Affiliated to Bharathiar University) | Master of Computer Applications | Department of Computer Science | Year 20__ - 20__

Laboratory Record

NAME: _____ ROLL No.: _____

SEMESTER: _____ YEAR: _____

SUBJECT: _____

Certified Bonafide Record of Work Done by Mr / Ms _____

Staff In Charge: _____ HOD: _____

Place: _____ Date: _____

University Register No.: _____

Submitted for the University Practical Examination held on _____

Internal Examiner: _____ External Examiner: _____

Index

S.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
		MySQL (SQL)		
1		Banking System		
2		Online Reservation System		
3		Personal Information		
4		Student Mark Processing System		
		MongoDB (NoSQL)		
5		E-Commerce Product Catalog System		
6		CURD Operations		

S.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
7		Built-in Functions		
8		Sorting, Indexing & Filtering		
9		Create a Collection & Find Document		
10		Stock Exchange		

Program 1: Banking System

Primary Key, Unique Key creation

```
create table bank(accno int primary key, cust_name varchar(15), email
varchar(100) unique, phonenumber varchar(15) unique);
desc bank;
```

Field	Type	Null	Key	Default	Extra
accno	int	NO	PRI	NULL	
cust_name	varchar(15)	YES		NULL	
email	varchar(100)	YES	UNI	NULL	
phonenumber	varchar(15)	YES	UNI	NULL	

```
insert into bank values(17071,'velan','velan01@gmail.com','9788934500');
insert into bank values(17072,'anu','anupriya@gmail.com','6734517000'),
(17073,'shanthi','sanu21@gmail.com','7890012345');
select * from bank;
```

accno	cust_name	email	phonenumber
17071	velan	velan01@gmail.com	9788934500
17072	anu	anupriya@gmail.com	6734517000
17073	shanthi	sanu21@gmail.com	7890012345

Unique key constraint violation

```
insert into bank values (17074,'arun','anupriya@gmail.com','6734517000');
ERROR 1062 (23000): Duplicate entry 'anupriya@gmail.com' for key 'bank.email'
```

Inserting NULL value in unique key field

```
insert into bank values (17074,'arun',NULL,'9894238341');
insert into bank values (17075,'padma','padma@gmail.com',null);
insert into bank values (17076,'preethi',null,null);
select * from bank;
```

accno	cust_name	email	phonenumber
17071	velan	velan01@gmail.com	9788934500
17072	anu	anupriya@gmail.com	6734517000
17073	shanthi	sanu21@gmail.com	7890012345
17074	arun	NULL	9894238341
17075	padma	padma@gmail.com	NULL
17076	preethi	NULL	NULL

Foreign Key creation

```
create table loandetails(loanid int primary key, accno int, loanamt int,
foreign key (accno) references bank(accno));
desc loandetails;
```

Field	Type	Null	Key	Default	Extra
loanid	int	NO	PRI	NULL	
accno	int	YES	MUL	NULL	
loanamt	int	YES		NULL	

```
insert into loandetails values(101,17071,50000);
insert into loandetails values(102,17074,60000);
insert into loandetails values(103,17077,45000);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`db1`.`loandetails`, CONSTRAINT `loandetails_ibfk_1`
FOREIGN KEY (`accno`) REFERENCES `bank` (`accno`))
select * from loandetails;
```

loanid	accno	loanamt
101	17071	50000
102	17074	60000

Combining two table data using Joining function

```
select b.cust_name,l.loanid,l.loanamt from bank b inner join loandetails as l
on b.accno=l.accno;
```

cust_name	loanid	loanamt
velan	101	50000
arun	102	60000

Program 2: Online Reservation System

Table 1: Table creation, value insertion & viewing data and its field

Table Name: Customer | Primary key: c_id

```
create table customer (c_id int primary key,c_name varchar(10),email
varchar(25),phonenumber varchar(10));
insert into customer values(1071,'sethu','sethu@gmail.com','7634529800');
insert into customer values(1072,'mani','mani@gmail.com','9843978876'),
(1073,'jancy','jancy15@gmail.com','9342156567');
select * from customer;
```

c_id	c_name	email	phonenumber
1071	sethu	sethu@gmail.com	7634529800
1072	mani	mani@gmail.com	9843978876
1073	jancy	jancy15@gmail.com	9342156567

```
desc customer;
```

Field	Type	Null	Key	Default	Extra
c_id	int	NO	PRI	NULL	
c_name	varchar(10)	YES		NULL	
email	varchar(25)	YES		NULL	
phonenumber	varchar(10)	YES		NULL	

Table 2: Creation & description of the table

Table Name: Vehicle | Primary key: vid

```
create table vehicle(vid int primary key,v_type
enum('car','van','bus'),capacity int);
desc vehicle;
```

Field	Type	Null	Key	Default	Extra
vid	int	NO	PRI	NULL	
v_type	enum('car','van','bus')	YES		NULL	
capacity	int	YES		NULL	

Value insertion and enum data type violation

```
insert into vehicle values(1001,'car',4);
insert into vehicle values(1200,'van',10);
insert into vehicle values(1201,'bus',52);
insert into vehicle values(1201,'bike',1);
ERROR 1265 (01000): Data truncated for column 'v_type' at row 1
```

Viewing vehicle table data

```
select * from vehicle;
```

vid	v_type	capacity
1001	car	4
1200	van	10
1201	bus	52

Table 3: Creation & setting check integrity to the field start_datetime

Table Name: reserve | Primary key: r_id | Foreign key: c_id, vid

Check integrity constraint assigned to the start_datetime column.

```
create table reserve(r_id int primary key,c_id int,vid int,start_datetime
datetime check(start_datetime >='2025-10-25'),end_datetime datetime,palce
varchar(15),foreign key(c_id) references customer(c_id),foreign key(vid)
references vehicle(vid));
desc reserve;
```

Field	Type	Null	Key	Default	Extra
r_id	int	NO	PRI	NULL	
c_id	int	YES	MUL	NULL	
vid	int	YES	MUL	NULL	
start_datetime	datetime	YES		NULL	
end_datetime	datetime	YES		NULL	
place	varchar(15)	YES		NULL	

```
insert into reserve values(1,1071,1200,'2025-10-26 06:00:00','2025-10-26
09:00:00','salem');
insert into reserve values(2,1073,1201,'2025-10-25 10:30:00','2025-10-27
06:00:00','delhi');
insert into reserve values(3,1072,1200,'2025-10-20 10:00:00','2025-10-25
08:00:00','madurai');
ERROR 3819 (HY000): Check constraint 'reserve_chk_1' is violated.
select * from reserve;
```

r_id	c_id	vid	start_datetime	end_datetime	place
1	1071	1200	2025-10-26 06:00:00	2025-10-26 09:00:00	salem
2	1073	1201	2025-10-25 10:30:00	2025-10-27 06:00:00	delhi

Program 3: Personal Information

```
create table personal_data(username varchar(10),emailid varchar(25)
unique,UG_University varchar(25));
desc personal_data;
```

Field	Type	Null	Key	Default	Extra
username	varchar(10)	YES		NULL	
emailid	varchar(25)	YES	UNI	NULL	
UG_University	varchar(25)	YES		NULL	

```
insert into personal_data
values('sumathi','sumathi15@gmail.com','bharathiyar'),
('ravi','ravi23@gmail.com','bharathidasan'),
('keerthi','keerthi030@gmail.com','bharathiar');
insert into personal_data values('sumathi','suma2104@gmail.com','periyar');
insert into personal_data values('kumar','kumar101@gmail.com','bharathiar');
select * from personal_data;
```

username	emailid	UG_University
sumathi	sumathi15@gmail.com	bharathiyar

username	emailid	UG_University
ravi	ravi23@gmail.com	bharathidasan
keerthi	keerthi030@gmail.com	bharathiar
sumathi	suma2104@gmail.com	periyar
kumar	kumar101@gmail.com	bharathiar

Using LIKE operator

```
SELECT * FROM personal_data WHERE emailid LIKE '%@gmail.com%';
```

Result: all 5 rows (5 rows in set).

```
SELECT * FROM personal_data WHERE username LIKE '_a%';
```

username	emailid	UG_University
ravi	ravi23@gmail.com	bharathidasan

```
SELECT * FROM personal_data WHERE username LIKE '%a_';
```

username	emailid	UG_University
kumar	kumar101@gmail.com	bharathiar

```
SELECT * FROM personal_data WHERE UG_University LIKE '%a_';
```

Result: all 5 rows (5 rows in set).

```
SELECT * FROM personal_data WHERE UG_University LIKE '%ar';
```

username	emailid	UG_University
sumathi	sumathi15@gmail.com	bharathiyar
keerthi	keerthi030@gmail.com	bharathiar

username	emailid	UG_University
sumathi	suma2104@gmail.com	periyar
kumar	kumar101@gmail.com	bharathiar

```
SELECT * FROM personal_data WHERE UG_University NOT LIKE '_h%';
```

username	emailid	UG_University
sumathi	suma2104@gmail.com	periyar

GROUP BY with aggregate functions in USER table

```
select * from user;
```

Iduser	username	place	invested
1001	anu	cbe	NULL
1002	priya	ooty	10000
1003	monika	cbe	10000
1005	kiru	ooty	10000

```
select count(*) from user;        -- 4
select sum(invested) from user;   -- 30000
select avg(invested) from user;   -- 10000.0000
```

```
update user set invested=5000 where Iduser=1003;
select * from user;
```

Iduser	username	place	invested
1001	anu	cbe	NULL
1002	priya	ooty	10000
1003	monika	cbe	5000

Iduser	username	place	invested
1005	kiru	ooty	15000

```
select place, avg(invested) from user group by place;
```

place	avg(invested)
cbe	5000.0000
ooty	10000.0000

```
select place,min(invested) from user group by place;
```

place	min(invested)
cbe	5000
ooty	10000

```
select place,sum(invested) from user group by place;
```

place	sum(invested)
cbe	5000
ooty	25000

ORDER BY and LIMIT

```
select * from personalinfo order by studentname asc;
```

rollno	studentname	fathername	occupation	email	created_at
25MCA04	Abi	Arunachalam	Manager	abi01@gamil.com	2025-09-02 14:47:13
25MCA01	Anu	Murugan	Manager	anupriya@gmail.com	2025-09-02 14:17:57
25MCA02	Banupriya	Balaji	salesman	banu@gmail.com	2025-09-02 14:19:55
25MCA06	Mani	Senthil	Teacher	mani007@gmail.com	2025-09-02 14:55:33

rollno	studentname	fathername	occupation	email	created_at
25MCA03	Prakash	Kumar	Clerk	prakash21@gmail.com	2025-09-02 14:21:00

```
select * from personalinfo order by studentname asc limit 1;
```

rollno	studentname	fathername	occupation	email	created_at
25MCA04	Abi	Arunachalam	Manager	abi01@gamil.com	2025-09-02 14:47:13

```
select * from personalinfo order by studentname desc limit 1;
```

rollno	studentname	fathername	occupation	email	created_at
25MCA03	Prakash	Kumar	Clerk	prakash21@gmail.com	2025-09-02 14:21:00

HAVING clause in EMP table

```
select * from emp;
```

eid	ename	age	edu
1	sindhu	25	mca
2	kavi	26	mca

```
insert into emp values(3,'rithu',25,'mca'),(4,'raju',26,'mca'),
(5,'ganesh',25,'mca'),(6,'priya',26,'mca');
select * from emp;
```

eid	ename	age	edu
1	sindhu	25	mca
2	kavi	26	mca
3	rithu	25	mca
4	raju	26	mca
5	ganesh	25	mca

eid	ename	age	edu
6	priya	26	mca

```
select edu,count(eid)as num_employees from emp group by edu having
count(eid)>3;
```

edu	num_employees
mca	6

```
select age,count(eid)as num_employees from emp group by age having
count(eid)>=3;
```

age	num_employees
25	3
26	3

Program 4: Student Mark Processing System

Table creation

```
create table studentdata(rollno varchar(8) primary key,studentname
varchar(15),java int,adbms int,cn int,os int,d_net int,total int,percentage
int);
desc studentdata;
```

Field	Type	Null	Key	Default	Extra
rollno	varchar(8)	NO	PRI	NULL	
studentname	varchar(15)	YES		NULL	
java	int	YES		NULL	
adbms	int	YES		NULL	
cn	int	YES		NULL	
os	int	YES		NULL	

Field	Type	Null	Key	Default	Extra
d_net	int	YES		NULL	
total	int	YES		NULL	
percentage	int	YES		NULL	

Insert values to table

```
insert into studentdata
(rollno,studentname,java,adbms,cn,os,d_net)values('25mca01','arun',50,57,67,7
8,50);
insert into studentdata
(rollno,studentname,java,adbms,cn,os,d_net)values('25mca02','banu',65,57,69,6
0,48),('25mca03','chitra',78,67,56,46,98);
select * from studentdata;
```

rollno	studentname	java	adbms	cn	os	d_net	total	percentage
25mca01	arun	50	57	67	78	50	NULL	NULL
25mca02	banu	65	57	69	60	48	NULL	NULL
25mca03	chitra	78	67	56	46	98	NULL	NULL

Update total, percentage column

```
update studentdata set total=java+adbms+cn+os+d_net;
update studentdata set percentage=total/5;
select * from studentdata;
```

rollno	studentname	java	adbms	cn	os	d_net	total	percentage
25mca01	arun	50	57	67	78	50	302	60
25mca02	banu	65	57	69	60	48	299	60
25mca03	chitra	78	67	56	46	98	345	69

Logical operators using WHERE clause

```
select rollno,java from studentdata where java between 45 and 65;
```

rollno	java
25mca01	50
25mca02	65

```
select rollno,java,d_net from studentdata where java>45 and d_net>45;
```

rollno	java	d_net
25mca01	50	50
25mca02	65	48
25mca03	78	98

```
select rollno,cn,os from studentdata where cn>=60 or os>=60;
```

rollno	cn	os
25mca01	67	78
25mca02	69	60

```
select rollno,studentname from studentdata where total not between 300 and
350;
```

rollno	studentname
25mca02	banu

```
select rollno,studentname from studentdata where total <> 302;
```

rollno	studentname
25mca02	banu
25mca03	chitra

```
select rollno,studentname,total from studentdata where total=299;
```

rollno	studentname	total
25mca02	banu	299

Program 5: Design an E-Commerce product catalog system using MongoDB as a storage engine and insert values

Step 1: Open mongoshell

```
show databases;
```

Output: admin 40.00 KiB, config 108.00 KiB, local 72.00 KiB, mongodbVSCodePlaygroundDB 40.00 KiB.

Step 2: show databases; and shift to the newly created database named e_commerce

```
use e_commerce
switched to db e_commerce
```

Step 3: Insert values to the new collection named product and view the created collections

```
db.products.insertMany([
  { name: "Laptop", category: "Electronics", price: 1200, stock: 50 },
  { name: "Mouse", category: "Electronics", price: 25, stock: 200 },
  { name: "Keyboard", category: "Electronics", price: 75, stock: 150 },
  { name: "Desk Chair", category: "Furniture", price: 300, stock: 30 },
  { name: "Coffee Table", category: "Furniture", price: 150, stock: 40 },
  { name: "Bookcase", category: "Furniture", price: 200, stock: 20 },
  { name: "T-Shirt", category: "Apparel", price: 20, stock: 500 },
  { name: "Jeans", category: "Apparel", price: 60, stock: 100 }
]);
```

Step 4: View the data in Table view (e_commerce > products, 8 documents)

name	category	price
Laptop	Electronics	1200
Mouse	Electronics	25
Keyboard	Electronics	75
Desk Chair	Furniture	300
Coffee Table	Furniture	150
Bookcase	Furniture	200
T-Shirt	Apparel	20
Jeans	Apparel	60

Step 5: View data in mongoshell

```
db.products.find()
{ _id: ObjectId('68f9afa582de359b62abe9ef'), name: 'Laptop', category:
'Electronics', price: 1200, stock: 50 }
{ _id: ObjectId('68f9afa582de359b62abe9f0'), name: 'Mouse', category:
'Electronics', price: 25, stock: 200 }
{ _id: ObjectId('68f9afa582de359b62abe9f1'), name: 'Keyboard', category:
'Electronics', price: 75, stock: 150 }
```

Program 6: Perform basic CURD operations (Create, Update, Read and Delete) functions for the product catalog

Step 1: Create collection collegeshop in database e_shopping

```
use e_shopping
switched to db e_shopping
db.createCollection("collegeshop")
{ ok: 1 }
```

Step 2: Insert one document in collegeshop

```
db.collegeshop.insertOne({item:"notebook",qty:50})
{ acknowledged: true, insertedId: ObjectId('68e72b45771f57bccf5daf3b') }
```

Insert many documents in collegeshop

```
db.collegeshop.insertMany([{item:"ballpointpen",qty:100},
{item:"inkpen",qty:60}])
{ acknowledged: true, insertedIds: { '0':
ObjectId('68e72b8e771f57bccf5daf3c'), '1':
ObjectId('68e72b8e771f57bccf5daf3d') } }
```

Step 3: Read documents from collection using find()

```
db.collegeshop.find({qty:{$gt : 15 }})
{ _id: ObjectId('68e72b45771f57bccf5daf3b'), item: 'notebook', qty: 50 }
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100 }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'inkpen', qty: 60 }
```

Read documents from collection using findOne()

```
db.collegeshop.findOne({item:"ballpointpen"})
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100 }
```

Step 4: To find documents where qty is greater than 30 and less than 100

```
db.collegeshop.find({ qty: { $gt: 30, $lt: 100 } })
{ _id: ObjectId('68e72b45771f57bccf5daf3b'), item: 'notebook', qty: 50 }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'inkpen', qty: 60 }
```

To find documents where qty is greater than or equal to 60 and less than or equal to 100

```
db.collegeshop.find({ qty: { $gte: 60, $lte: 100 } })
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100 }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'inkpen', qty: 60 }
```

Step 5: To Update one document from the collection collegeshop

```
db.collegeshop.updateOne(
  { item: "notebook" },
  { $set: { "category": "paperstuff" } }
)
```

To view the data updation use find()

```
db.collegeshop.find();
{ _id: ObjectId('68e72b45771f57bccf5daf3b'), item: 'notebook', qty: 50,
category: 'paperstuff' }
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100 }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'inkpen', qty: 60 }
```

Updating the other two documents one by one using updateOne

```
db.collegeshop.updateOne({ item: "ballpointpen" }, { $set: { "category":
"writingstuff" } })
db.collegeshop.updateOne({ item: "inkpen" }, { $set: { "category":
"writingstuff" } })
```

Step 6: To UpdateMany documents (all documents that match the specified filter criteria)

```
db.collegeshop.updatemany({category:"writingstuff"},{$set:
{satus:"available"}})
TypeError: db.collegeshop.updatemany is not a function
```

Proceed with step 5 to update the documents and continue with step 6, then the output:

```
db.collegeshop.updateMany({category:"writingstuff"},{$set:
{satus:"available"}})
{ acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2,
upsertedCount: 0 }

db.collegeshop.find()
{ _id: ObjectId('68e72b45771f57bccf5daf3b'), item: 'notebook', qty: 50,
category: 'paperstuff' }
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100,
category: 'writingstuff', satus: 'available' }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'inkpen', qty: 60,
category: 'writingstuff', satus: 'available' }
```

Step 7: To replaceOne this document with a new one

```
db.collegeshop.replaceOne({item:"inkpen"},
{item:"blueinkpen",qty:70,category:"writingstuff",status:"available"})
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1,
upsertedCount: 0 }
```

The item inkpen data is replaced by using the above query.

```
db.collegeshop.find()
{ _id: ObjectId('68e72b45771f57bccf5daf3b'), item: 'notebook', qty: 50,
category: 'paperstuff' }
{ _id: ObjectId('68e72b8e771f57bccf5daf3c'), item: 'ballpointpen', qty: 100,
category: 'writingstuff', satus: 'available' }
{ _id: ObjectId('68e72b8e771f57bccf5daf3d'), item: 'blueinkpen', qty: 70,
category: 'writingstuff', status: 'available' }

db.collegeshop.insertOne({item:"colourbox",qty:55,category:"writingstuff",sta
tus:"available"})
{ acknowledged: true, insertedId: ObjectId('68e74dff771f57bccf5daf3e') }
```

Deleting a single document

```
db.collegeshop.deleteOne({item:"colourbox"})
{ acknowledged: true, deletedCount: 1 }
```

Deleting many documents

```
db.collegeshop.deleteMany({category:"writingstuff"})
{ acknowledged: true, deletedCount: 2 }
db.collegeshop.find()
```

Program 7: Built-in functions in MongoDB

Step 1: Use e_shopping database, insert values to s_mark collection

```
use e_shopping
db.s_mark.insertOne({_id:1,sname:"Arun",java:76,cn:73,os:98});
db.s_mark.insertMany([{_id:2,sname:"banu",java:67,cn:56,os:76},
{_id:3,sname:"chitra",java:89,cn:67,os:54},
{_id:4,sname:"dharshan",java:73,cn:45,os:69}])
```

Step 2: Aggregate function $sum

```
db.s_mark.aggregate([
  { $addFields: { totalmark: { $sum: ["$java", "$cn", "$os"] } } }
])
```

Result of $sum:

```
{ _id: 1, sname: 'Arun', java: 76, cn: 73, os: 98, totalmark: 247 }
{ _id: 2, sname: 'banu', java: 67, cn: 56, os: 76, totalmark: 199 }
```

Step 3: Aggregate functions $add, $multiply, $divide

```
db.s_mark.aggregate([
  { $addFields: {
      totalmark: { $add: ["$java", "$cn", "$os"] },
      percentage: { $multiply: [ { $divide: [{ $add: ["$java", "$cn", "$os"]
}, 300] }, 100 ] }
  } }
])
```

Result of $add, $multiply, $divide:

```
{ _id: 1, sname: 'Arun', java: 76, cn: 73, os: 98, totalmark: 247,
percentage: 82.33333333333334 }
{ _id: 2, sname: 'banu', java: 67, cn: 56, os: 76, totalmark: 199,
percentage: 66.33333333333333 }
```

Step 4: Update document with field edu_status

```
db.s_mark.updateOne( {_id:1}, { $set: {"edu_status": "regular"}})
db.s_mark.updateOne( {_id:2}, { $set: {"edu_status": "intern"}})
db.s_mark.updateOne( {_id:3}, { $set: {"edu_status": "regular"}})
db.s_mark.updateOne( {_id:4}, { $set: {edu_status: "intern"}})
```

Step 5: Find documents in Table view and in mongoshell

_id	sname	java	cn
1	Arun	76	73
2	banu	67	56
3	chitra	89	67
4	dharshan	73	45

```
db.s_mark.find()
{ _id: 1, sname: 'Arun', java: 76, cn: 73, os: 98, edu_status: 'regular' }
{ _id: 2, sname: 'banu', java: 67, cn: 56, os: 76, edu_status: 'intern' }
{ _id: 3, sname: 'chitra', java: 89, cn: 67, os: 54, edu_status: 'regular' }
{ _id: 4, sname: 'dharshan', java: 73, cn: 45, os: 69, edu_status: 'intern' }
```

Step 6: Aggregate function $match - Filters documents based on specified conditions, similar to the WHERE clause in SQL.

```
db.s_mark.aggregate([{$match: {edu_status:"regular"}}])
{ _id: 1, sname: 'Arun', java: 76, cn: 73, os: 98, edu_status: 'regular' }
{ _id: 3, sname: 'chitra', java: 89, cn: 67, os: 54, edu_status: 'regular' }
```

Step 7: Aggregate function $match with condition

```
db.s_mark.aggregate([{ $match: { edu_status: "intern", os: { $gte: 70 } } }])
{ _id: 2, sname: 'banu', java: 67, cn: 56, os: 76, edu_status: 'intern' }
```

Step 8: Without aggregate function, documents filtered using find()

```
db.s_mark.find({edu_status:"intern"})
{ _id: 2, sname: 'banu', java: 67, cn: 56, os: 76, edu_status: 'intern' }
{ _id: 4, sname: 'dharshan', java: 73, cn: 45, os: 69, edu_status: 'intern' }
```

Step 9: Aggregate function $group

```
db.s_mark.aggregate([
  { $group: {
      _id: "$edu_status",          // group key
      avg_java: { $avg: "$java" },
      avg_cn: { $avg: "$cn" },
      avg_os: { $avg: "$os" },
      count: { $sum: 1 }           // how many students in each group
  } }
])
```

Result of $group:

```
{ _id: 'intern', avg_java: 70, avg_cn: 50.5, avg_os: 72.5, count: 2 }
{ _id: 'regular', avg_java: 82.5, avg_cn: 70, avg_os: 76, count: 2 }
```

Step 10: $count

```
db.s_mark.aggregate([ { $match: { "edu_status": "regular" } }, { $count:
"totalregularstudents" } ])
{ totalregularstudents: 2 }

db.s_mark.aggregate([ { $match: { "edu_status": "intern" } }, { $count:
"Total_Intern_Students" } ])
{ Total_Intern_Students: 2 }

db.s_mark.estimatedDocumentCount()               // 4
db.s_mark.countDocuments({})                     // 4
db.s_mark.countDocuments({edu_status:"intern"})  // 2
```

Step 11: Including and excluding fields using $project and $limit

```
db.product.aggregate([
  { $project: { "_id":0, "name": 1, "category": 1, "stock": 1 } },
  { $limit: 2 }
])
```

Result of $project:

```
{ name: 'Laptop', category: 'Electronics', stock: 50 }
{ name: 'Mouse', category: 'Electronics', stock: 200 }
```

Excluding the "_id" field (student collection):

```
db.student.aggregate([{ $project: {"_id":0,}}])
{ firstname: 'adhi', lastname: 'ravi', age: 16, email: 'adhiravi@gmail.com' }
{ firstname: 'adhi', lastname: 'mani', age: 12, email: 'dhivya@gmail.com' }
{ firstname: 'balu', lastname: 'velu', age: 15, email: 'baluv@gmail.com' }
```

Error because MongoDB does not allow mixing inclusion and exclusion in the same $project except for the _id field.

```
db.student.aggregate([{ $project:
{"_id":0,"firstname":1,"lastname":1,"age":0,"email":1}}])
Error[Location31254]: Invalid $project :: caused by :: Cannot do exclusion on
field age in inclusion projection
```

Only exclusions:

```
db.student.aggregate([{ $project: { _id: 0, age: 0 } }])
{ firstname: 'adhi', lastname: 'ravi', email: 'adhiravi@gmail.com' }
{ firstname: 'adhi', lastname: 'mani', email: 'dhivya@gmail.com' }
```

Adding new fields using $project:

```
db.student.aggregate([{ $project: { "fullName": { $concat: ["$firstname", "
", "$lastname"] } } }])
{ _id: 1, fullName: 'adhi ravi' }
{ _id: 2, fullName: 'adhi mani' }
{ _id: 3, fullName: 'balu velu' }
```

Program 8: Sorting, filtering and Index in MongoDB

Step 1: Use e_shopping database, insert values to product collection

```
db.product.insertMany([
  { name: "Laptop", category: "Electronics", price: 1200, stock: 50 },
  { name: "Mouse", category: "Electronics", price: 25, stock: 200 },
  { name: "Keyboard", category: "Electronics", price: 75, stock: 150 },
  { name: "Desk Chair", category: "Furniture", price: 300, stock: 30 },
  { name: "Coffee Table", category: "Furniture", price: 150, stock: 40 },
  { name: "Bookcase", category: "Furniture", price: 200, stock: 20 },
  { name: "T-Shirt", category: "Apparel", price: 20, stock: 500 },
  { name: "Jeans", category: "Apparel", price: 60, stock: 100 }
]);
```

Step 2: Filter the documents based on category-wise and price-wise

```
db.product.find({category:"Electronics", price: { $lt: 100 }})
{ _id: ObjectId('68f8983adabcf8634ae16e10'), name: 'Mouse', category:
'Electronics', price: 25, stock: 200 }
{ _id: ObjectId('68f8983adabcf8634ae16e11'), name: 'Keyboard', category:
'Electronics', price: 75, stock: 150 }
```

Step 3: Sort the documents based on price field in ascending order

```
db.product.find().sort({ price: 1 })
```

Step 4: Filter the documents based on category: Furniture and apply descending order based on stock field

```
db.product.find({ category: "Furniture" }).sort({ stock: -1 });
```

Step 5: Filter and sort the documents based on category, price and name

```
db.product.find({ category: "Electronics", price: { $gt: 50 } }).sort({ name:
1 });
{ _id: ObjectId('68f8983adabcf8634ae16e11'), name: 'Keyboard', category:
'Electronics', price: 75, stock: 150 }
{ _id: ObjectId('68f8983adabcf8634ae16e0f'), name: 'Laptop', category:
'Electronics', price: 1200, stock: 50 }
```

Step 6: Before applying Index

```
db.product.getIndexKeys()
[ { _id: 1 } ]
db.product.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Step 7: After applying Index

```
db.product.createIndex({ category: 1, price: 1 });
category_1_price_1
db.product.getIndexKeys()
[ { _id: 1 }, { category: 1, price: 1 } ]
db.product.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { category: 1, price: 1 }, name: 'category_1_price_1' }
]
```

Program 9: Create a Collection and Document and perform the following

a) Find a document by id

b) Find a user by email

c) Find a list of all users with the same firstname

d) Find all users who are more than 12 years old

d) Find all users who are more than 12 years old

```
db.student.find({age: {$gt:12}})
{ _id: 1, firstname: 'adhi', lastname: 'ravi', age: 16, email:
'adhiravi@gmail.com' }
{ _id: 3, firstname: 'balu', lastname: 'velu', age: 15, email:
'baluv@gmail.com' }
```

c) Find a list of all users with the same firstname

```
db.student.aggregate([
  { $group: {
      _id: "$firstname",         // group by firstName field
      count: { $sum: 1 },        // count how many users have that name
      users: { $push: "$$ROOT" } // keep the full user documents
  } },
  { $match: {
      count: { $gt: 1 }          // only include names that appear more than
once
  } }
]);
```

```
{
  _id: 'adhi',
  count: 2,
  users: [
    { _id: 1, firstname: 'adhi', lastname: 'ravi', age: 16, email:
'adhiravi@gmail.com' },
    { _id: 2, firstname: 'adhi', lastname: 'mani', age: 12, email:
'dhivya@gmail.com' }
  ]
}
```

b) Find a user by email

```
db.student.findOne({ email: { $regex: "@gmail\\.com$", $options: "i" } });
{ _id: 1, firstname: 'adhi', lastname: 'ravi', age: 16, email:
'adhiravi@gmail.com' }

db.product.findOne({ email: { $regex: "@kgisliim\\.ac\\.in$", $options: "i" }
});
{ _id: 2, username: 'Shruthi', age: 19, email: 'shruthi@kgisliim.ac.in' }
```

a) Find a document by id

```
db.student.findOne({ _id:2 });
{ _id: 2, firstname: 'adhi', lastname: 'mani', age: 12, email:
'dhivya@gmail.com' }
```

Program 10: Use real time data set of Stock exchange by import CSV data

Step 1: Create new database sample and collection named stock

In MongoDB Compass: Create Database > Database Name: sample, Collection Name: stock > Create Database.

Step 2: Choose Collection menu from the menubar and select Import Data

Step 3: Choose stock_data.csv from the open dialog

Import settings: collection sample.stock | Import file: stockdata_test.csv | Delimiter: Comma | Ignore empty strings: checked | Stop on errors: unchecked | Field types: productcode = String, profit = Double, growth = Double.

Step 4: Imported data in table view

productcode	profit	growth
stk001	0.72	0.5
stk002	0.45	-0.04
stk003	1.25	0.32
stk004	-0.12	-0.06
stk005	1.05	0.23
stk006	0.67	0.4

Step 5: Find all the stocks where the profit is over 0.5

```
db.stock_data.find({ profit: { $gt: 0.5 } });
{ _id: ObjectId('68f9045717f0a1ecb33f0d5e'), productcode: 'stk001', profit:
0.72, growth: 0.15 }
{ _id: ObjectId('68f9045717f0a1ecb33f0d60'), productcode: 'stk003', profit:
1.25, growth: 0.32 }
{ _id: ObjectId('68f9045717f0a1ecb33f0d62'), productcode: 'stk005', profit:
1.05, growth: 0.23 }
{ _id: ObjectId('68f9045717f0a1ecb33f0d63'), productcode: 'stk006', profit:
0.67, growth: 0.4 }
```

Step 6: Find all the stocks with negative growth

```
use sample
switched to db sample
db.stock_data.find({ growth: { $lt: 0 } });
{ _id: ObjectId('68f9045717f0a1ecb33f0d5f'), productcode: 'stk002', profit:
0.45, growth: -0.04 }
{ _id: ObjectId('68f9045717f0a1ecb33f0d61'), productcode: 'stk004', profit:
-0.12, growth: -0.06 }
```

Command line import method

Import CSV file

```
C:\Program Files\MongoDB\Tools\100\bin>mongoimport --db sample --collection
stock --type csv --file D:\ADBMS\Practical\stockdata_test.csv --headerline
2025-10-27T13:39:43.136+0530   connected to: mongodb://localhost/
2025-10-27T13:39:43.140+0530   6 document(s) imported successfully. 0
document(s) failed to import.
```

Import JSON file

```
C:\Program Files\MongoDB\Tools\100\bin>mongoimport --db sample --collection
stock --file "D:\ADBMS\Practical\output_stockdata1.json"
2025-10-27T14:00:00.404+0530   connected to: mongodb://localhost/
2025-10-27T14:00:00.408+0530   6 document(s) imported successfully. 0
document(s) failed to import.
```

Export JSON file format

```
C:\Program Files\MongoDB\Tools\100\bin>mongoexport --db=sample --
collection=stock --out="D:\ADBMS\Practical\output_stockdata.json"
2025-10-27T13:45:30.285+0530   connected to: mongodb://localhost/
2025-10-27T13:45:30.292+0530   exported 6 records

C:\Program Files\MongoDB\Tools\100\bin>mongoexport --db sample --collection
stock --out "D:\ADBMS\Practical\output_stockdata1.json"
2025-10-27T13:47:34.767+0530   connected to: mongodb://localhost/
2025-10-27T13:47:34.772+0530   exported 6 records
```

Export CSV file format

```
C:\Program Files\MongoDB\Tools\100\bin>mongoexport --db sample --collection
stock --out "D:\ADBMS\Practical\output_stock.csv"
2025-10-27T13:53:47.054+0530   connected to: mongodb://localhost/
2025-10-27T13:53:47.061+0530   exported 6 records
```